

CASE STUDY · HEALTHCARE SAAS · ML IN PRODUCT

Predicting Coding Rework Before It Happens

An ML feature embedded in athenahealth's coding platform that cut rework by 8–12% — and changed how coders work, every day.

8–12%

REDUCTION IN REWORK

50K+

NOTES IN TRAINING SET

Near real-time

SCORING IN PRODUCT

Pilot → GA

ROLLOUT PATH

Rework was draining coder hours and delaying customer revenue.

WHAT THE PRODUCT DOES

Our SaaS platform converts clinicians' notes into billing codes. When the codes are wrong, the chart bounces back for rework — wasting coder time and delaying revenue for the practices we serve.

WHY IT WAS UNDER-MANAGED

Leaders knew rework was elevated in certain specialties, but coders had no signal of which specific notes were risky. Every chart looked the same in the queue — until it bounced back.



Coder time, twice over

Every reworked chart consumed coder capacity twice — once on the original pass, once on the correction.



Delayed customer revenue

Bounced charts pushed back claim submission, which pushed back when our customers got paid.



No targeting signal

Coders treated every note alike. Without a risk signal, extra scrutiny was spent randomly — not where it mattered.

We had the data to predict rework — we just hadn't tried.

THE PROPOSAL

Instead of one more dashboard, build a model that flags risky notes before they cause rework — and put it inside the tool coders already use.



TARGET

Rework: a chart whose initial codes were rejected and required a coder correction.

Definition co-validated with Ops



UNIT OF PREDICTION

The note. We score notes, not coders — keeping the focus on the work, not the worker.

Important for coder buy-in



PRIMARY METRIC

Reduction in rework rate during pilot vs. matched baseline period.

Operational impact, not just AUC



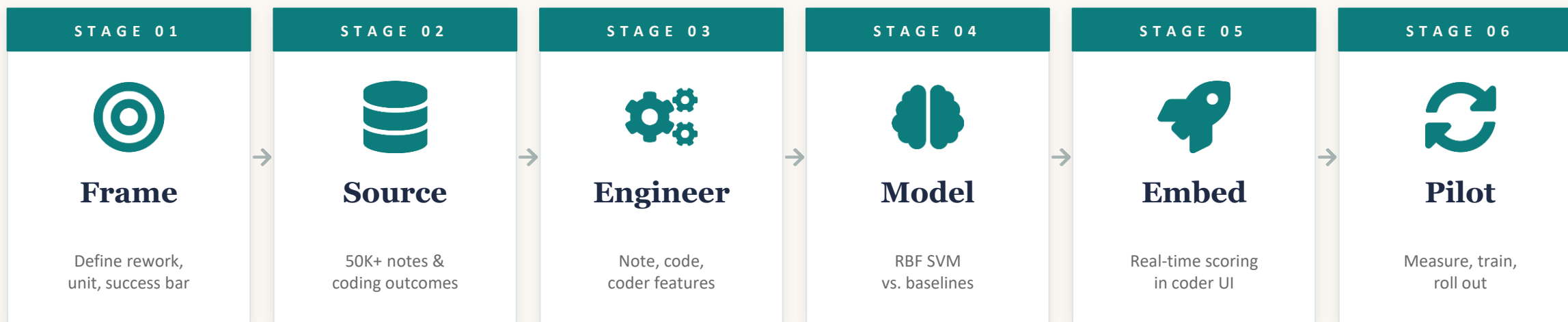
MODEL METRICS

Recall on rework class + precision-at-threshold; alert volume kept manageable for coders.

Tuned for the workflow, not the leaderboard

Six stages — from labeling problem to embedded product feature.

I owned the full lifecycle, partnering with Ops on labels, with Product & Engineering on the integration, and with GTM on the pilot rollout.



WHY THIS APPROACH WORKED

An ML model alone wouldn't move rework — embedding it in the coder's actual queue did. Workflow integration was as important as model accuracy.

50K+ notes, a clean rework label, no leakage across encounters.

DATA SOURCES



Clinical notes

50K+ notes from the platform — note metadata, length, specialty, and structure



Coding records

Codes assigned (diagnoses, procedures), modifiers, E/M levels, downstream rework events



Coder activity

Coder identity, tenure, specialty mix, historical accuracy and volume

LABELING THE TARGET

1

Identify rework event

Mapped the system event that captures a downstream correction.

2

Validate with Ops

Excluded test notes, system glitches, and edge cases.

3

Build binary label

Each note: 1 = rework, 0 = clean.



PHI & LEAKAGE GUARDRAILS

Splits respected encounter boundaries (no patient leakage across train/test). All work in the secure environment under standard PHI controls.

CLASS IMBALANCE · Rework is the minority class — addressed via class-weighted loss and threshold tuning, not naive resampling.

Three feature families, each grounded in coder intuition.

Every feature came from a hypothesis a coder or auditor would recognize — if a domain expert couldn't explain why it might predict rework, we didn't include it.



Note characteristics

EXAMPLES

- Note length (very short and very long both signal risk)
- Specialty / encounter type
- Structural attributes (sections, completeness)

WHY: Length and structure are proxies for documentation complexity.



Code complexity

EXAMPLES

- Number of diagnosis & procedure codes
- E/M level (visit complexity tier)
- Presence and count of modifiers

WHY: Higher complexity = more places for a coding decision to go wrong.



Coder history

EXAMPLES

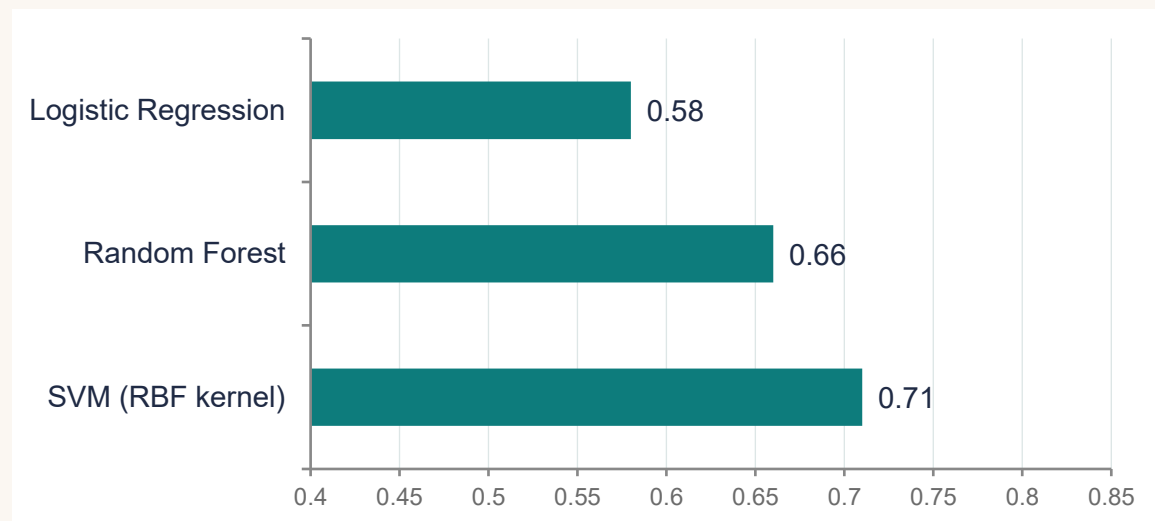
- Historical rework rate (note-level, not punitive)
- Specialty mix and volume
- Tenure

WHY: Captures real variance in code-decision patterns across coders.

DOMAIN VALIDATION · Top features matched what auditors said mattered — building stakeholder trust before the model ever shipped.

RBF-SVM won the bake-off — non-linear interactions mattered.

BAKE-OFF — RECALL ON REWORK CLASS



Logistic regression couldn't capture the interaction between code complexity, note length, and coder history. Random forest closed part of the gap — RBF-SVM closed the rest.

WHY RBF-SVM WAS THE RIGHT FIT



Captures non-linear interactions

Risk spikes when complexity + length + coder pattern combine — RBF kernel handles that natively.



Strong on moderate dataset sizes

50K+ samples is in SVM's sweet spot, where it's competitive with tree-based models.



Class-weighted loss for imbalance

Penalizes missing rework cases more than false alarms — aligned with our operational cost.



Cross-validated tuning

Grid-searched C and gamma with k-fold CV; held-out test set for an honest performance read.

PRINCIPLE · Let the data shape the model choice. Non-linearity in the residuals = more powerful kernel earned its place.

Hyperparameters tuned for recall — threshold tuned for the workflow.

HYPERPARAMETER SEARCH

HYPERPARAMETER	RANGE	STRATEGY
C (regularization)	0.1 → 100	log-grid
gamma (RBF width)	0.001 → 1	log-grid
class_weight	balanced / custom	imbalance
decision threshold	tuned on PR-curve	workflow
k-fold CV	5-fold	stratified

Stratified 5-fold CV on training set; final unbiased read on a fully held-out test set.

CHOOSING THE THRESHOLD



Recall first, but not at any cost

Catching rework matters most — but flagging too many notes turns the alert into background noise.



Alert volume capped per coder

We capped flagged notes at a level coders said they could meaningfully review.



Co-designed with Ops

Walked Ops through the precision–recall curve so they could see the trade-off, not just inherit a number.



Outcome: focus over alarm fatigue

Coders trusted the flags because they were rare enough to mean something.

OUTPUT · A risk score per note + a binary flag at the chosen threshold — both surfaced in the coder's queue UI.

From notebook to coder queue — scoring embedded in the live workflow.



Note arrives

New chart enters the coder queue



Features

Signals computed in near real-time



Score

RBF-SVM returns rework probability



Flag

High-risk notes marked in UI



Coder acts

Extra scrutiny on flagged notes



Built INTO the existing tool

Not a side dashboard. Scores surface as small flags in the queue coders already work in — zero context-switching.



GTM partnership for adoption

Partnered with GTM to pilot with select customers, train coders on what the flag means, and address concerns about scoring fairness.

INTEGRATION PRINCIPLE · *If the model isn't where the work happens, it doesn't change the work.*

Coders had real concerns. We addressed them, head-on.

Some coders worried the model was "grading" them. Their concerns were legitimate — and shaped how we framed and rolled out the feature.

THE CONCERN

"This is going to grade me."

OUR RESPONSE

We score notes, not coders.

The unit of prediction is the note. Coder history is one signal among many, never surfaced individually.

THE CONCERN

"What if I disagree with the flag?"

OUR RESPONSE

Flags are advisory, not enforcement.

Coders kept full authority to code as they saw fit. The flag was a heads-up, not a gate.

THE CONCERN

"How will I trust it?"

OUR RESPONSE

Show the wins, then let it earn trust.

Shared concrete examples where flags caught real rework. Adoption grew as coders saw fewer charts bouncing back.

Pilot vs. matched baseline — rework dropped, throughput held.

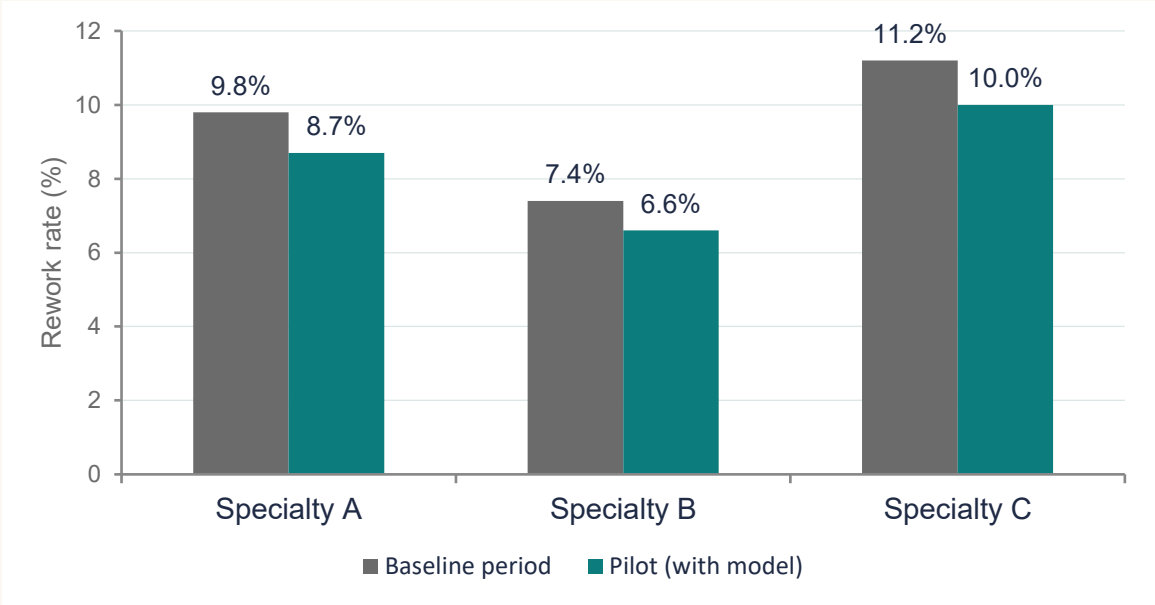
THE PILOT DESIGN

We rolled the feature out to a subset of customers and coders, and compared their rework rate to a matched baseline period plus similar groups without the feature.

WHAT WE TRACKED

- Rework rate — primary outcome
- Coding throughput — to confirm flags didn't slow people down
- Alert volume per coder — to watch for fatigue
- Coder feedback — qualitative, weekly

REWORK RATE — PILOT vs. BASELINE



Illustrative — directional values shown for confidentiality.



HEADLINE IMPACT

8–12% relative reduction in rework cases across pilot groups, with throughput unchanged and coder feedback positive.

The scoreboard.



REWORK REDUCTION

8–12%

across pilot groups vs. matched baseline period



NOTES IN TRAINING SET

50K+

with encounter-aware splits to prevent leakage



SCORING IN PRODUCT

Real-time

embedded directly in the coder queue UI



ROLLOUT PATH

Pilot → GA

broad rollout after pilot proved consistent gains

QUALITATIVE WINS

Coders reported they could focus extra review time on the right notes · Feature became a standard part of the coding workflow · Established pattern for future ML embedded in the SaaS product.

What worked, what I'd do differently.

WHAT WORKED



Embedding the model in the workflow

A flag in the existing queue beat any standalone dashboard. Adoption came from zero context-switching.



Domain-validated features

Top features matched what auditors said mattered — that earned trust before the model ever shipped.



Threshold tuned with operators, not in a vacuum

Co-designing alert volume with Ops prevented alert fatigue and made the feature credible from day one.

WHAT I'D DO DIFFERENTLY



Add lightweight NLP on note text

We used structured features only. Modest text-derived features could have lifted recall further.



Benchmark calibrated tree-based models

Calibrated gradient-boosted trees might match SVM with better probability outputs and easier deployment.



Run a true A/B test from day one

Matched-baseline comparison was solid, but a randomized rollout would give cleaner causal evidence.

ML in the workflow, not ML on the side.

01

Predict where the work happens.

Embedding scoring directly in the coder UI was the difference between an interesting model and a feature that actually changed behavior.

02

Score the work, not the worker.

Framing predictions at the note level kept coder trust intact and made adoption sustainable.

03

Tune for the operator, not the leaderboard.

Recall + alert volume were the real metrics — AUC alone wouldn't have kept the feature in production.

Thank you · Questions?